Optimizing University Timetabling - a MIP and Heuristic Approach for the University of Edinburgh

Ashe Raymond-Barker Dmitri Goverdovsky Michael Walshe Stavros Mitas

April 2026

1 Introduction

In this report, we examine the University of Edinburgh timetabling problem. Teaching is currently scheduled from Monday to Friday, 9am to 6pm, with as little core (whole-class or compulsory) teaching as possible taking place on Wednesday afternoons. To improve the experience of both students and staff, the timetabling team proposed two potential changes to the current teaching schedule:

- reducing teaching hours to Monday-Friday, 9am-5pm;
- eliminating all teaching on Friday afternoons (after 12pm).

This report assesses the feasibility of both of these proposed schedules, as well as a third, more compressed option in which teaching takes place from 9am-5pm, Monday to Thursday, and from 9am-12pm on Fridays.

In addition, we evaluate which of these schedules produces the most effective timetables for students and for the Estates department. In particular, we address two key questions:

- How does changing the teaching schedule affect clashes between core teaching events?
- What impact do these changes have on room and space utilisation?

In Section 2, we review existing approaches to university timetabling. To construct feasible timetables under the proposed schedules, we use two distinct methods: an exact Mixed-Integer Programming (MIP) approach and a heuristic Simulated Annealing (SA) approach. The problem specifics and data we used are described in Section 3. The methods are described in Section 4. We evaluate the timetables produced under each proposed scenario using several key metrics, including the number of teaching clashes, room utilisation, and room capacity violations. Based on these results, we assess the potential impact of changes to the university's teaching schedule and make recommendations to the timetabling team. Our main findings and recommendations are presented in Section 12.

2 Literature Review

The University Timetabling Problem is a well-documented and researched NP-hard optimization problem [2, 9, 10]. Aguirre Lam et. al distinguish between two distinct problems: exam timetabling and course timetabling (UCTT) [2]. Course timetabling problems can be divided further into post-enrolment based timetabling and curriculum-based timetabling [10]. The former has constraints determined by student enrolment data, whereas the latter has constraints dependent on a curriculum that is specified by the university. This project is treated as a post-enrolment-based timetabling problem, since the goal is to find the optimal allocation of courses into time slots and rooms that match the class sizes and provide feasible student schedules.

The UCTT problem has received considerable attention in the academic literature [6, 15]. Its prominence is further reflected in initiatives such as the International Timetabling Competition (ITC), which brings together researchers and institutions to develop and evaluate highly effective solution methods [12, 13]. In this literature review, we outline the current state of the art in university course timetabling. While exact mathematical programming in combination with sufficiently powerful commercial solvers can be used to find optimal solutions to the problem, the large size, combinatorial

nature and NP-completeness of the timetabling problem frequently make such approaches impossible within reasonable time-frames [15]. A substantial body of research has therefore focused on developing heuristic approaches that aim to find near-optimal solutions efficiently [13]. In this section, we consider four key methods: an exact approach (Mixed-Integer Programming), and two heuristic approaches: Simulated Annealing and Genetic Algorithms.

2.1 Exact Solution: Mixed-Integer-Programming Approaches

Mixed Integer Programming is frequently used for university timetabling because it offers exact optimization and certain optimality [5, 9, 15]. The winning entry in the 2019 ITC employed a MIP based approach [13]. A key advantage of MIP solvers is that, besides producing feasible timetables, they also provide lower bounds on the best possible objective value. This makes it possible to measure how close a timetable is to optimal [5]. Assignment decisions can be represented with binary variables and the particular university requirements can be translated into linear constraints. In this setting, hard constraints are enforced explicitly, while soft constraints are typically incorporated into the objective function as penalties. The solver then finds a feasible solution subject to the hard constraints while optimizing the solution with respect to the soft constraints [5].

2.1.1 Decomposition into Smaller Problems

In large-scale problems, it is often the case that a single MIP exceeds practical solve time limits [15]. Many timetabling practices treat the institution in question as a single entity resulting in tractability problems. Literature suggests that decomposition is the most effective way to improve solvability [9]. In some cases, such as at the University of Edinburgh, the university-wide problem can be split geographically [20]. The University of Edinburgh analyses room utilisation by campus, suggesting that certain campuses can be treated as autonomous scheduling zones with degree programmes typically scheduled exclusively at these campuses. Furthermore, the current methodology at the university prioritizes activities into timetabling zones as requested by schools, to prevent excessive travel for staff and students between scheduled locations campuses [19].

Further reductions in complexity are also possible. Lach et. al note that, after decomposing geographically, the scheduling process itself can be separated into a two-step method, through splitting the decision into a time assignment stage, followed by a room assignment stage [9]. They find solutions to the second stage that are strongly comparable with the best approaches on the benchmark instances described in [13]. An alternative decomposition, used by Cacchiani et. al [4], involves splitting the objective function into separate parts, considering each part as an individual problem, then summing corresponding optimal values. They used integer linear programming to find the minimum for room penalties first, and then constructed the calender. Adding the optimal of the first step to the optimal result of the second yields a theoretical lower bound for the timetabling problem. Cachianni et. al report that using this approach yields results that improve on benchmark results of the 2007 ITC [12].

2.2 Heuristic Approximation Approaches

Exact methods offer the advantage of provable optimality, however the UCTT problem is fundamentally NP-hard, and thus becomes computationally infeasible for larger problems [3, 5, 10, 17]. To solve these issues, heuristic and meta heuristic approaches have been extensively used, with success in benchmark testing and practical employability. Notably, the winners of the 2019 ICT employed a meta heuristic algorithm, outperforming competitive MIP formulations [5]. Building on the hierarchical constraint classification established by [3], meta heuristic frameworks typically use a weighted penalty function within the objective to prioritize the satisfaction of hard constraints, which ensures schedule feasibility, while minimizing soft constraint violations to optimize the overall quality of the resulting timetable. Drawing on the conceptual framework of [17], modern approaches are designed to navigate complex solutions by balancing intensification, which focuses on high-precision local search to improve existing assignments, and diversification, which utilizes stochastic tools to escape local optima and explore new regions of the search space.

2.2.1 Simulated Annealing Approaches

A specific heuristic approach that has seen recent success is simulated annealing (SA), placing first on the first and second milestones of the 2019 ICT, as well as winning third place on the final phase and the first prize in the open source category [16]. The method draws inspiration from the metallurgical process of annealing, with the probabilistic technique allowing the search to escape local optima by occasionally accepting 'worse' moves, such as a schedule change that temporarily increases student clashes, with a probability that decreases as the 'temperature' of the system cools [7]. Thus, the over-arching benefit of SA is the flexibility to explore a wider range of solutions, increasing the chances of arriving at a near optimal result.

2.2.2 Genetic Algorithm Approaches

An alternative heuristic method that could be applied to this problem is genetic algorithms [1, 11, 14]. Genetic algorithms begin by generating an initial population of feasible solutions and evaluating their objective values. A new population is then produced by selecting the strongest candidates and combining their characteristics in a way analogous to biological crossover. The idea is that recombining favourable traits from high-performing solutions may lead to even stronger candidates. This process is repeated over successive generations, with the best solutions at each stage being used to produce the next population. Although genetic algorithms are an interesting and widely studied class of heuristic methods and often prove successful in other domains [8, 18], they have achieved comparatively limited success in the ITC challenges [12, 13]. For this reason, while we considered them as a possible approach, we decided not to explore them further in this project.

3 Data and Problem Setup

3.1 Data Description and Cleaning

The raw data provided by the University's timetabling team comprised five files covering the 2024/25 academic year (Table 1). Together these describe 28,198 teaching events across 4,177 modules, delivered by 24 departments on eight campuses, and allocable to 697 rooms with capacities ranging from 1 to 458 seats.

File	Description
Room and Room Types	Room capacities, buildings, and specialist types
Programme-Course	Programme-to-module mappings with compulsory flags
Student Programme Module Event	Individual student enrolments to events
Event Module Room	Events with module, duration, and room requirements
DPT Data	Departmental planning and timetable structures

Table 1: Primary datasets used in the timetabling model (all for 2024-25).

Several issues required cleaning before the data could be used in our models. Event sizes contained non-numeric entries which were turned to numeric values, with missing sizes set to zero. Campus labels were inconsistent between datasets, for example the rooms file used "Bioquarter" while the events file used "BioQuarter". We standardised these. The weeks field, which records which teaching weeks an event runs, used a mix of comma-separated lists and hyphenated ranges. To fix this, we parsed both formats into uniform comma-separated integers. For Semester 1 only weeks 9 to 19 were retained, and for Semester 2 only weeks 26 to 36, matching the University's standard teaching periods. Existing room assignments and governance data were removed. Online events were flagged but retained, as they still require a time slot even though they do not occupy a physical room.

After cleaning, the five raw files were consolidated into three input tables used by both the MIP and SA models. `Cleaned_Event_Module_Room`, links every event to its module, department, duration, expected attendance, required room type, campus, semester, and teaching weeks. `Cleaned_Rooms_and_RoomTypes`, giving each room's capacity, building, campus, and specialist type. Finally, `Cleaned_Programme_Courses`, mapping 1,452 programmes to their modules and indicating whether each is compulsory. This last table, combined with the student enrolment data, allows us to identify programme-level clash pairs: two events clash when they share students enrolled on the same pro-

gramme and year of study, and are scheduled in overlapping time slots. This makes our formulation a *post-enrolment* timetabling problem, using actual student-programme-event enrolment data rather than purely a curriculum structure.

3.2 Campus Decomposition and Time Discretisation

The University of Edinburgh operates across eight campuses, and in practice each campus is timetabled largely independently. We exploit this structure by decomposing the problem into campus sub problems, which reflects operational practice and greatly improves computational tractability. Our primary test campuses are Easter Bush (2,575 events, 33 rooms) and BioQuarter (677 events, 29 rooms), as well as Kings Buildings (5,365 events, 121 rooms). These campuses were chosen because they exhibit characteristics representative of the university-wide problem: diverse room types, specialist space constraints, and multi-programme clash potential.

For BioQuarter, an additional campus filter was applied beyond department filtering alone. The four BioQuarter departments (Edinburgh Medical School, School of Population Health Sciences, School of Neurological and Cardiovascular Sciences, and School of Health in Social Science) also deliver events on the Central campus. Without filtering by event campus, their combined event count would be far larger and would include events assigned to Central rooms. Kings Buildings was subdivided by department group: Mathematics, Chemistry and Physics, and Engineering, Geosciences and Biology. These department groups were then assigned specific buildings within the Kings Buildings Campus. This was done to manage test case sizes at that campus. We also included a test version of Kings Buildings to include all departments, as well as central campus and university-wide tests. Only the SA algorithm was used for these, since the test set would be too great for MIP to solve due to computational power restrictions. However, a further MIP test was carried out in which time assignments for Kings Buildings events were solved separately for the subdivided department groups, after which rooms were assigned using a combined model.

The teaching day was divided into uniform time slots of either 30 or 60 minutes. Under a 9 am to 6 pm window with 30 minute slots, each day has 18 slots and under 60 minute slots, each day has 9. An event of duration d minutes occupies $\lceil d/s \rceil$ consecutive slots, where s is the slot length. The 30 minute granularity allows, for example, a 90 minute lecture to occupy exactly 3 slots rather than rounding up to 2 full hours, providing greater scheduling flexibility. Both granularities were tested to assess their impact on solution quality.

3.3 Scenarios and Problem Instances

Each campus-semester combination was tested under four scenario variants designed to answer the timetabling team's core questions:

Timetabling Scenarios Considered:

1. **9–6, Mon–Fri (current schedule):** teaching runs 9 am–6 pm Monday to Friday.
2. **9–5, Mon–Fri:** teaching runs 9 am–5 pm Monday to Friday.
3. **9–6, short Friday:** teaching runs 9 am–6 pm Monday to Thursday, and 9 am–12 pm on Friday.
4. **9–5, short Friday:** teaching runs 9 am–5 pm Monday to Thursday, and 9 am–12 pm on Friday.

Combined with 30-minute and 60-minute slot granularities, this yields eight configurations per campus-semester pair. Table 2 summarises the problem instances. This progression allows us to validate the SA against optimal or near-optimal MIP solutions on tractable instances, before applying it at scales where exact methods pose time and computation power issues.

4 Methodology: MIP Approach

We first modelled the timetabling problem using a decomposition-based mixed-integer optimization framework implemented in Pyomo and solved with Gurobi. The main reason for using decomposition was tractability. A single university-wide formulation would have required the solver to choose times

Campus / Subset	Sem	Events	Rooms	Configs	Method(s)
Easter Bush	1	1,554	33	6	MIP, SA
Easter Bush	2	1,021	33	6	MIP, SA
BioQuarter	1	376	29	6	MIP, SA
BioQuarter	2	301	29	6	MIP, SA
KB – Mathematics	1	672	121	6	MIP, SA
KB – Chem / Physics	1	625	121	6	MIP, SA
KB – Eng / Geo / Bio	1	1,608	121	6	MIP, SA
KB – All depts	1	2,800	121	6	SA
Central	1	7,573	339	6	SA
University-wide	1	28,198	697	6	SA

Table 2: Problem instances tested. Event counts are per semester. The MIP was used for smaller instances; the SA was applied to all instances including larger ones where exact methods become intractable.

and rooms for all activities simultaneously, producing a model that was too large to solve reliably within practical time limits particularly with our limited computational power. Instead, we decomposed the problem in two ways. First, we split the university into relatively autonomous scheduling zones, primarily by campus and, where helpful, by building group or subject area. Second, within each zone, inspired by the work of Lach et. al [9], we separated the timetabling task into a time-assignment stage and a room-assignment stage. This follows the broad decomposition ideas discussed in the literature, but was tailored to the structure of the University of Edinburgh estate and teaching patterns.

This decomposition was also useful for answering the two policy questions posed by the scheduling team. Rather than changing the model itself, we changed the set of feasible teaching slots in each scenario. In particular, the scenarios corresponding to a shorter teaching day and the removal of Friday afternoon teaching were represented by removing the relevant slots from the planning horizon. This allowed like-for-like comparisons across scenarios while keeping the same underlying optimization framework.

4.1 Stage 1: Time Assignment

The first stage assigns each event to a teaching day and start time. Currently the University of Edinburgh makes use of 60-minute time slots, but while exploring the data, we noticed that there were a significant number of non-whole hour events (e.g there are c. 1,700 90 minute events). We anticipated that switching to 30-minute time slots would lead to less wasted time for such events, and possibly allow us more space to compress teaching into the timetabling teams reduced schedules. We therefore tested timetabling under both 30 and 60-minute time slots to examine whether there were any benefits to 30 minute slots.

Event durations were converted into the corresponding number of slots, and feasible start times were restricted so that no event could extend beyond the end of the teaching day. In this way, policy scenarios such as shortening the teaching day by one hour or removing Friday afternoon teaching were imposed directly through the feasible start-time set.

The hard constraints in the time-assignment model were designed not only to produce a valid timetable, but also to guarantee that the later room-assignment stage would remain feasible. In particular, we imposed aggregate room-availability constraints by room-type pool. For each week, day, slot, and compatible room-type pool, the number of events scheduled to occupy that slot could not exceed the number of rooms available in that pool. This means that the time-assignment model already respects the supply of specialist spaces in aggregate, even before specific rooms are chosen. We also allowed certain substitutions that reflect operational practice: for example, teaching studios were treated as interchangeable with general teaching rooms, and general teaching events were allowed to use any suitable teaching space.

The time-assignment stage was itself solved in two phases. In Phase 1, we solved a pure feasibility model containing only the hard constraints, with objective value zero. This quickly determines whether

a feasible timetable exists for a given scenario. In Phase 2, we added the soft constraints and used the Phase 1 solution as a warm start. This was computationally preferable to solving the full model with hard and soft constraints from the outset, because it separated the difficult question of feasibility from the secondary question of timetable quality.

The principal soft constraint penalised overlaps between events that were marked as whole-class for the same programme-year. For pairs of such events, we added a penalty whenever their assigned time intervals overlapped on the same day. The weight of the penalty increased with the number of shared whole-class programme assignments and the number of common teaching weeks. This objective is quadratic because it contains products of binary assignment variables, so the optimization phase is more accurately described as a mixed-integer quadratic programme. In practical terms, however, it remains part of the same MIP-based modelling framework.

4.2 Stage 2: Room Assignment

Once event times had been fixed, the second stage assigned individual rooms. This stage is defined over event occurrences rather than event patterns: if an event takes place in several weeks, each weekly occurrence receives a room. Room compatibility was based on room type, with the same substitution rules as in Stage 1. The room-assignment objective penalised capacity shortfall. For an event of size n_o assigned to a room of capacity c_r, the penalty was given by:

$$((n_o - c_r)_+)^2,$$

where $(\cdot)_+$ denotes the positive part ($\max\{0, (n_o - c_r)\}$). Using a squared penalty assigns a much higher cost to severe under-sizing than to small capacity mismatches. As a result, our programme prefers timetables with many small capacity shortfalls over those with only a few but much larger mismatches. This reflects the practical importance of avoiding large-scale room capacity mismatches.

Since the time-assignment stage already enforced aggregate room-type feasibility, the room-assignment model was much easier to solve. In practice, the computational difficulty was concentrated almost entirely in Stage 1. Once a feasible time assignment had been found, the room-assignment stage became relatively straightforward.

This motivated the testing of an alternative timetabling approach for the Kings Buildings, in which the more difficult time assignment problems were solved separately for the subdivided department groups, before the simpler room assignment problem was solved using a combined model. It was hypothesised that the increased room flexibility provided by this approach would significantly reduce room capacity constraint violations.

4.2.1 Implications of the Decomposition

The main advantage of the decomposition is tractability. It makes the model solvable on realistic university instances and allows rapid testing of alternative policy scenarios. The trade-off is that the sequential approach may sacrifice global optimality relative to a fully integrated one-stage model. In an integrated formulation, the solver could trade off time and room decisions simultaneously. In the sequential formulation used here, event times are fixed before rooms are chosen, so the best achievable objective value of the decomposed approach is, in general, at least as large as that of the corresponding one-stage model. We therefore interpret the decomposition as a pragmatic compromise: it sacrifices some theoretical optimality in exchange for a method that is transparent, computationally manageable, and suitable for scenario testing in a real institutional setting.

Computational limitations played an important role in our model design. The most powerful computer available for this project had 24GB of RAM, while timetabling methods in the literature are often implemented on machines with substantially greater memory capacity, in some cases up to 256GB of RAM [5]. Consequently, we were unable to apply these methods at full scale exactly as described in prior work. To ensure tractability and obtain stable solutions within available computational resources, we introduced further reductions in problem size and additional decomposition. This was a pragmatic compromise that sacrificed some modelling generality, but enabled us to generate results capable of informing the university's questions about shortening the teaching day and removing Friday afternoon

teaching.

5 Methodology: Simulated Annealing

To complement the MIP, we implemented a simulated annealing algorithm following the same two-stage decomposition. The primary purpose was to provide a scalable fallback for larger instances where the MIP exceeded available memory.

The SA mirrors the two-stage MIP structure. In the time-assignment stage, the algorithm begins from a randomly generated feasible solution and at each iteration reassigns a single event to a different feasible day–slot pair. The change in objective value is evaluated incrementally, maintaining an index of clash pairs per event and evaluating only affected pairs. This reduces the per-iteration cost from $O(|P|)$ to $O(\deg(e))$, which was essential for achieving sufficient iterations on larger instances.

The Stage 1 objective extends the MIP clash penalty with an additional weighted room-type violation term, penalising solutions in which more events of a given type are scheduled simultaneously than there are compatible rooms available. This term has no counterpart in the MIP, where room-pool feasibility is enforced as a hard constraint. Because the SA operates on one event at a time, it cannot enforce these aggregate constraints exactly, and the penalty term takes the search towards feasibility without guaranteeing it. This is a known limitation of single-move structures on constrained problems, and the consequences are discussed in Section 11.

The room-assignment stage takes the fixed time assignment from Stage 1 and assigns rooms via a greedy best-fit-decreasing initialisation, sorting occurrences by event size and assigning each to the smallest compatible conflict-free room. This is then refined by a second SA phase using the same squared capacity deviation objective as the MIP Stage 2, augmented by a double-booking penalty to discourage operationally infeasible assignments. Full details of the SA approach can be found in Appendix B.

6 Evaluation Metrics

To evaluate and compare the solutions produced by the MIP and SA across all scenarios, we report a consistent set of metrics that capture both optimization quality and student experience under a given schedule. Three primary metrics correspond directly to the objective functions of the two-stage model.

Performance Measures Used in the Model:

- **Programme-level clashes** measure the weighted count of whole-class event pairs that overlap in time and share at least one programme. This is weighted by the number of shared programmes and common teaching weeks. As the Stage 1 objective, it reflects the extent to which students are forced to miss compulsory teaching.
- **Capacity deviation** is the total shortfall across all room assignments, where a shortfall occurs when attendance exceeds room capacity. This is the Stage 2 objective and captures how well rooms are matched to event sizes. For interpretability, we report a linear measure, but implement a quadratic penalty to discourage large mismatches.
- **Overcrowded events** count the number of events assigned to rooms with insufficient capacity. This complements the aggregate capacity deviation by indicating how many individual events are affected.

  Note: These metrics are calculated over event occurrences, not just distinct events. Thus, if an event occurs in 11 teaching weeks, any associated penalty or violation is counted 11 times, once for each occurrence.

We also report three further metrics that help illustrate the distribution of teaching under the different scheduling regimes and the effect changes to the schedule may have on room utilisation rate.

- **Percentage of teaching in the 5-6 pm slot:** measures the fraction of events scheduled in the final hour under the current 9 am-6 pm schedule, quantifying how much evening teaching exists

under each scenario.

- **Friday afternoon teaching load:** reports the number of events scheduled after 12 pm on Fridays, measuring progress toward eliminating Friday afternoon teaching.
- **Average room utilisation** is the proportion of available room-slots that are occupied across the teaching window. After observing that weeks with few events (such as weeks outside of teaching time) can significantly skew this average, we report the mean utilisation across all rooms on the given campus for the busiest week.

Understanding the proportion of teaching that occurs between 5-6 under the current schedule and what proportion occurs on Friday afternoons gives insight into the scale of the change we are looking to make.

In the next section, we report the values of these metrics under the current Monday-Friday 9-6 teaching arrangement to provide a baseline for the timetabling team to understand better how switching to a each of the proposed new teaching schedules would affect student experience and estate usage.

7 Baseline Results

7.1 Clashes and Capacity Mismatches

In this section, we describe the characteristics of a timetable produced under the current Monday-Friday 9 am-6 pm teaching schedule, which serves as the baseline for comparison across all subsequent scenarios. The timetable is obtained by solving the two-stage MIP across three campuses: Easter Bush, BioQuarter, and King's Buildings. Results are reported in Table 3.

Easter Bush and BioQuarter are each solved as a single integrated problem across both stages of the model, allowing all evaluation metrics to be reported directly. In contrast, King's Buildings is treated differently due to its scale. Attempting to solve the Stage 1 room assignment for KB resulted in memory issues, which is consistent with findings in literature. Even with 256GB of RAM, exact timetabling methods have encountered memory limitations on university-wide instances [5]. The first stage 1 (timeslot assignment) is decomposed into three departmental sub-groups to make computation possible: Chemistry/Physics, Engineering/Geography/Biology, and Mathematics, which are solved independently and programme-level clashes are reported separately for each before being summed to give a campus-wide total.

This decomposition introduces a limitation. Because Stage 1 is solved independently per sub-group, the aggregate room-type feasibility constraint is enforced only within each sub-group rather than across King's Buildings as a whole. In principle, two sub-groups could simultaneously schedule more events requiring a given room type than are available across the combined pool. Stage 2 addresses this by performing room assignment jointly across all KB events regardless of sub-group, using the full campus-wide room inventory. This ensures the final room assignments are feasible and double-booking free, even if the Stage 1 time assignments were made without full cross-campus room-type awareness. Metrics related to room allocation are therefore reported only at Stage 2, after the combined room assignment has been solved. We report the corresponding results for the Kings Buildings when departments have to schedule all their teaching within their own buildings in Appendix D, but we omit them here for clarity.

7.2 Distribution of Teaching

Two further metrics from Section 6, the percentage of teaching in the 5-6 pm slot and the Friday afternoon teaching load, are reported in full in Appendix C. We summarise the key findings here since show the distribution of teaching under 9 am-6 am and how much redistributions need to occur under the new schedule.

Under the current 9 am-6 pm schedule, teaching later in the day is a significant feature of the timetable. Between 6% and 10% of events at Easter Bush and King's Buildings are scheduled in the 5–6 pm slot, confirming that evening teaching is not a marginal occurrence but a structural feature of the current timetable. We also see that Friday afternoon teaching is substantial across all campuses. In particu-

	Programme-level clashes		Capacity deviation		Overcrowded events		Avg. room util. (%)	
Campus / stage	Sem 1	Sem 2	Sem 1	Sem 2	Sem 1	Sem 2	Sem 1	Sem 2
Easter Bush	0	0	1,227	2,231	52	46	69.58	76.76
BioQuarter	36	0	800	654	51	29	64.78	60.14
KB Stage 1: timeslot assignment								
Chem/phys	111	585	–	–	–	–	–	–
Eng/geo/bio	66	106	–	–	–	–	–	–
Maths	178	776	–	–	–	–	–	–
Campus Wide (Sum)	355	1,467	–	–	–	–	–	–
KB Stage 2: room assignment								
Post-room-assignment outcome	–	–	4,476	5,864	124	136	57.50	57.32

For King's Buildings (KB), Stage 1 reports violations after assigning timeslots to classes but before room assignment. Stage 2 reports room-assignment violations after fixing the Stage 1 timeslots.

Table 3: This table reports the key student experience evaluation metrics outlined in Section 6 under the current 9-6 Mon-Fri teaching schedule.

lar King's Buildings carrying by far the largest load, with over 1,700 events scheduled after 12 pm on Fridays in Semester 1. As such, compressing the timetables in either way the timetabling team has suggested will require significant rescheduling. We report on the feasibility of such a rescheduling in Section 8 and the impact of such changes on student experience in Section 10.

8 Feasibility of Proposed Timetable Changes

For the Easter Bush, BioQuarter, and King's Buildings campuses, *all proposed timetable changes are feasible.*
Specifically, our model shows that it is possible to:

- reduce teaching hours to 9am-5pm, Monday-Friday;
- eliminate teaching on Friday afternoons; and
- implement both changes simultaneously.

9 On the Effects of Switching to 30 Minute Time Slots

We also tested whether switching from 60-minute to 30-minute time slots improved solution quality. The motivation was that finer granularity allows events whose durations are not multiples of one hour such as 90-minute lectures to be placed without wasted slot time, potentially creating more scheduling flexibility and reducing clashes. Table 4 reports the two primary objectives under both slot sizes for the baseline scenario.

		Programme-level clashes		Capacity deviation	
Campus	Sem	30 min	60 min	30 min	60 min
Easter Bush	1	0	0	1,423	1,227
Easter Bush	2	0	0	2,280	2,231
BioQuarter	1	38	36	800	800
BioQuarter	2	0	0	850	654

Table 4: MIP results under 30-minute and 60-minute slots, baseline scenario (9 am–6 pm, Friday afternoon permitted).

The results show no consistent benefit from switching to 30-minute slots. At Easter Bush, clashes remain zero under both granularities and capacity deviation is marginally worse under 30-minute slots in both semesters. At BioQuarter, capacity deviation is slightly higher under 30-minute slots in

Semester 2 (850 vs 654), while the clash counts are essentially identical. This may appear counter-intuitive, since a finer time grid should in principle expand the feasible region and allow the solver more flexibility. We attribute the lack of improvement to a limitation of the two-stage decomposition rather than to any inherent lack of value in shorter slots. Under the sequential approach, the Stage 1 model must commit to time assignments before rooms are chosen. With 30-minute slots, the number of feasible start times per event roughly doubles, substantially increasing the size and complexity of the Stage 1 model. Within the same computational budget, the solver therefore explores a larger but harder search space and does not reliably find better solutions. A fully integrated one-stage formulation, or a decomposition with a different ordering, would be better placed to make use of the additional flexibility that 30-minute slots provide. Given these findings, all scenario comparisons in this report use 60-minute slots, which produce equal or better results within our computational constraints.

10 Impact of Proposed Schedules on Student Experience and Estate Management

The MIP produced feasible timetables under all three proposed teaching schedules at the Easter Bush, BioQuarter, and King's Buildings campuses. However, feasibility alone does not mean that the resulting timetables are equally desirable. Figure 1 - Figure 4 show a clear trade-off between a more compressed teaching week and timetable quality. The corresponding data are presented in Appendix D in table form.

10.1 Effect on Clashes Between Teaching Events

Figure 1 shows that the impact of timetable compression on clashes differs markedly by campus. At Easter Bush, clashes remain negligible under all scenarios. The baseline timetable is clash-free, and even under the most restrictive schedule only a small number of clashes are introduced (3 in Semester 1), with Semester 2 unaffected. This indicates that Easter Bush can accommodate both reduced hours and the removal of Friday afternoons with minimal impact on students.

At BioQuarter, the effect is more pronounced but still moderate. In Semester 1, clashes increase from 36 in the baseline to 55 under a 9am–5pm schedule, and to 84 under the fully compressed timetable. By contrast, Semester 2 remains largely unaffected. This suggests that while timetable compression is feasible, removing Friday afternoon teaching introduces a clearer deterioration than simply shortening the teaching day.

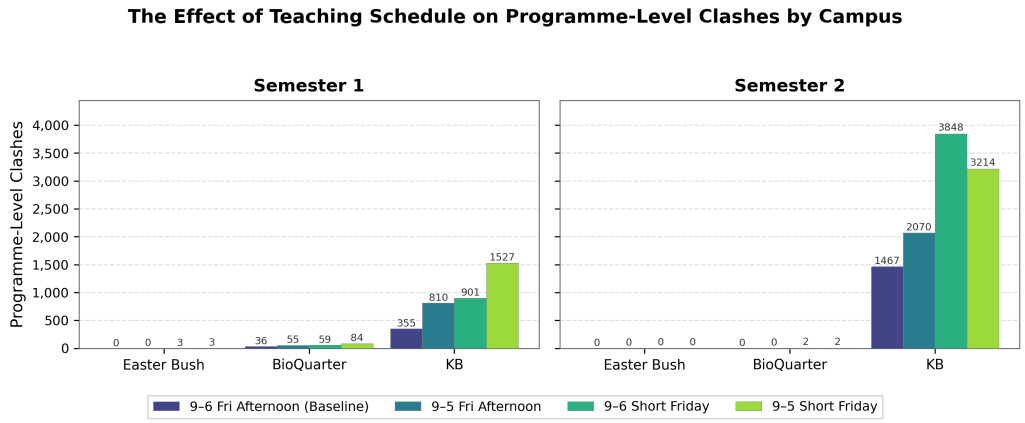

Figure 1: Programme-level clashes by campus and scenario (60-minute slots). KB Alt uses the KB-Total Stage 1 solution and therefore inherits its clash count.

King's Buildings is significantly more sensitive. Starting from an already high baseline (355 clashes in Semester 1 and 1,467 in Semester 2), all compressed schedules lead to substantial increases. In Semester 2, for example, clashes rise to over 3,800 when Friday afternoon teaching is removed. Across semesters, reducing teaching hours to 9am-5pm consistently results in fewer additional clashes than eliminating Friday afternoons.

10.2 Effect on Room and Space Utilisation

The capacity-based metrics are reported in Figure 2, Figure 3, and Figure 4. A consistent pattern across campuses is that compressing the teaching schedule is often accompanied by more overcrowded events and higher capacity deviation. Peak room utilisation remains largely unaffected between the four scenarios tested.

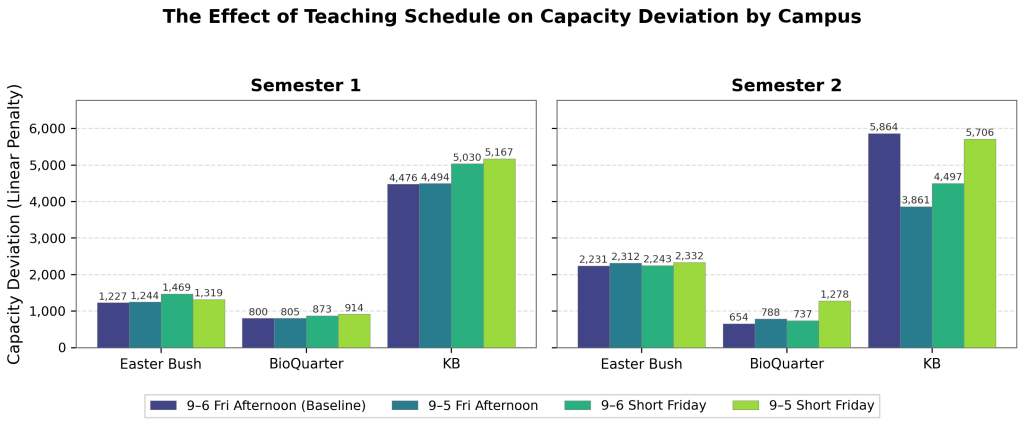

Figure 2: Linear capacity deviation by campus and scenario (60-minute slots).

Figure 2 shows that capacity deviation increases under most compressed scenarios. At smaller campuses, these increases are modest (e.g. Easter Bush rising from around 1,200 to at most 1,500), whereas BioQuarter shows larger relative increases in Semester 2 (from around 650 to over 1,200 under full compression). For King's Buildings under campus-wide room assignment we notice that in semester 1 capacity deviation changes little between the 9am-6pm and 9am-5pm schedules. For semester 2 however, we see a surprising results, that the capacity deviation is highest under the baseline teaching schedule. We expect this to be an artefact of the subject decomposition we used in creating our two-stage MIP in which under our decomposition, the solver cannot find the true optimal campus wide timetable - a key limitation of our approach.

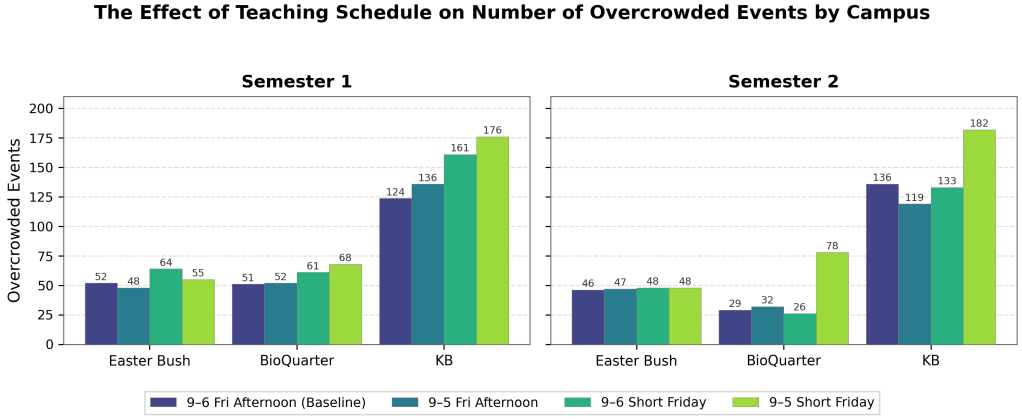

Figure 3: Number of overcrowded events by campus and scenario (60-minute slots).

A similar pattern is observed in Figure 3, where the number of overcrowded events increases across as the teaching week becomes more compressed. For example in Semester 1, the Kings Buildings sees an increase from 124 overcrowded events to 176. Similarly, for Semester 2 BioQuarter overcrowding rises from 29 to 78 events.

Considering capacity deviation alongside the number of overcrowded events provides further insight. The ratio $\frac{\text{capacity deviation}}{\text{overcrowded events}}$ can be interpreted as the mean shortfall per overcrowded event. For King's Buildings under the alternative room assignment, this is approximately 36.1 in Semester 1 and 43.1 in Semester 2 under the baseline schedule, but it falls across the compressed scenarios: to 33.0 and 32.4 under the 9 am–5 pm schedule, 31.2 and 33.8 under no Friday afternoon teaching, and 29.4

and 31.4 under the combined scenario. This suggests that timetable compression at King's Buildings under our model is not primarily making each overcrowded event worse on average. Instead, it creates more overcrowded events, while the average size of each shortfall tends to become smaller.

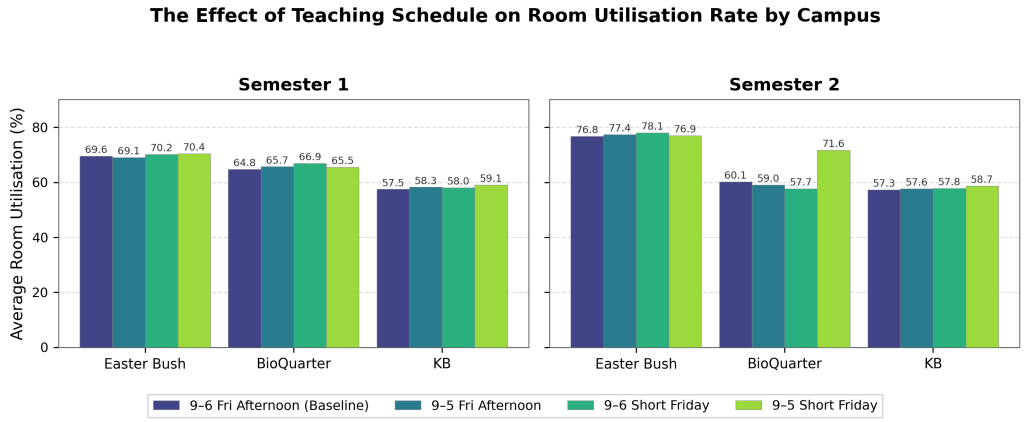

Figure 4: Average room utilisation by campus and scenario (60-minute slots).

Figure 4 shows that average room utilisation changes only slightly across scenarios. Even under compression, utilisation remains within a relatively narrow range - generally between around 55% and 75%. The greatest strain on room usage is seen in Easter Bush, which is expected as it is the smallest of each of the campuses we have considered.

Overall, these results highlight a clear trade-off. Compressing the teaching week leads to limited improvements in utilisation but results in both more frequent and more severe capacity violations. Among the alternatives, the 9 am–5 pm schedule appears to provide a more balanced outcome, whereas removing Friday afternoon teaching-particularly when combined with shorter teaching days-places the greatest strain on room capacity.

10.3 Campus Based Recommendations

- **Easter Bush:** Each of the three proposed schedules can be implemented on the Easter Bush campus with minimal impact on student experience as measured by our key metrics.
- **BioQuarter:** The BioQuarter campus can accommodate timetable compression, but with noticeable trade-offs. The 9am-5pm schedule provides a reasonable balance, with only moderate increases in clashes in programme-level teaching. However, removing Friday afternoon leads to a clearer deterioration in both clash counts and overcrowding, particularly in Semester 1.
- **King's Buildings:** Similarly to the BioQuarter, King's Buildings is particularly sensitive to the removal of Friday afternoon teaching. This change leads to a substantially larger increase in clashes compared to a shift to a 9am-5pm schedule. The 9am-5pm option results in fewer clashes and lower levels of overcrowding, making it the less disruptive of the two alternatives.
- Across all three campuses, overall pressure on teaching space remains relatively low. In particular, the mean peak room utilisation ratio stays below the 75% threshold preferred by the Estates department, indicating that the proposed schedules do not critically strain available capacity.

10.4 Replication of Results

The results presented in this report, including all tables and figures derived from the MIP model, are fully reproducible. The exact outputs can be regenerated using the Jupyter notebooks available in the project's GitHub repository (git@github.com:Khalva03/appliedOR.git). These notebooks contain the complete data processing pipeline, model implementation, and evaluation procedures used in this study.

11 On the Efficacy of our Simulated Annealing Approach

To complement the MIP, we implemented a Simulated Annealing heuristic following the same two-stage decomposition with comparable objective functions. The primary motivation was to provide a fallback approach for larger instances where the MIP would exceed available memory. Having evaluated the SA results, we conclude that the SA does not produce solutions of sufficient quality to inform the timetabling team's questions, and we do not use it as the basis for any of our recommendations. Table 5 summarises the comparison.

		Programme-level clashes		Capacity deviation		Overcrowded events		SA room-type violations	SA double bookings
Campus	Sem	MIP	SA	MIP	SA	MIP	SA		
Easter Bush	1	0	209	1,227	1,139	52	36	1,035	1
Easter Bush	2	0	81	2,231	2,158	46	34	328	0
BioQuarter	1	36	53	800	800	51	24	198	0
BioQuarter	2	0	6	654	696	29	22	95	0
KB – Total	1	355	30,402	4476	4,065	701	74	419	0
KB – Total	2	1467	30,242	5864	5,290	136	87	557	0

Table 5: MIP vs SA solution quality under the baseline scenario (9 am–6 pm, Friday afternoon permitted), 60-minute slots.

On the instances where both methods were applied, the SA performs substantially worse than the MIP on the primary objective of programme-level clashes. At Easter Bush, the MIP achieves zero clashes in both semesters while the SA produces 209 and 81 respectively. At BioQuarter the gap is smaller, the SA achieves 53 clashes in Semester 1 against the MIP's 36, and matches the MIP's zero clashes closely in Semester 2. King's Buildings is where the SA breaks down most severely: the MIP produces 355 and 1,467 clashes across the two semesters, while the SA produces 30,402 and 30,242. This is a gap of roughly two orders of magnitude. On the room assignment metrics, the SA is broadly comparable to the MIP at BioQuarter and King's Buildings, and marginally better on capacity deviation at Easter Bush. However, these tables must be interpreted with caution given the room-type violations we discuss next.

11.1 Room-Type Violations and the Single-Move Neighbourhood

A persistent and fundamental problem is the large number of room-type violations in the SA Stage 1 solution. At Easter Bush Semester 1 for example, violations remain fixed at 1,035 from early in the search and do not decrease further with iterations. Looking at the convergence logs confirms that the violation count plateaus well before the time limit is reached, while clashes continue to fall slowly. This behaviour is a direct consequence of the single-event neighbourhood structure used in Stage 1. To resolve a room-type violation, the algorithm must move one of the conflicting events to a different timeslot. However, because that event is typically involved in multiple clash pairs, any such move tends to increase the clash penalty by more than the violation penalty saves. The search therefore consistently prefers to reduce clashes and accept the violations, and no single-event move exists that improves both simultaneously.

Increasing the room-type violation penalty weight did not resolve this problem. Even at extreme values, the violation count remained unchanged, while the programme-level clash count worsened as the search became distorted by the inflated penalty. This confirms that the issue is structural rather than a matter of parameter tuning. The SA also produces one double booking in the Easter Bush Semester 1 room assignment. Unlike the MIP, which prevents double bookings by construction, the SA penalises them with a large weight, but cannot guarantee they are fully resolved before termination. The greedy initialisation for Stage 2 already contains this double booking, and the SA room phase does not find a conflict-free assignment within the time limit given the tight room supply at Easter Bush.

11.2 Performance on Large-Scale Instances

The one setting in which the SA produces output where the MIP cannot is the large-scale instances, where the MIP exhausts all available memory. However, the SA solutions for these instances contain very high room-type violations. This renders the solutions operationally infeasible, meaning they cannot be used as timetables without manual intervention to resolve the conflicts. The large-scale SA results (see appendix, can put the university wide results in there to show the SA can be activated) therefore demonstrate only that the algorithm can run at scale within a fixed time budget, not that it produces usable timetables. A reimplementation using compound neighbourhood moves that enforce room-type feasibility by construction would be required before the SA could serve as a genuine alternative for instances beyond the reach of the MIP.

We note that the difficulty encountered here is not specific to our implementation. The university course timetabling problem remains an active area of research, and developing heuristics that reliably produce feasible, state-of-the-art solutions has been the subject of dedicated international competitions with teams of professional researchers [13]. The purpose of our SA was to test whether a standard implementation could complement the MIP as a scalable fallback for the applied University of Edinburgh problem, and after critical evaluation, the answer is that it cannot without more sophisticated move operators. Developing a competitive heuristic for this problem is itself a research contribution, and appears to be beyond the scope of this project.

12 Recommendations and Future Work

12.1 Limitations for Joint Degree Programmes and Cross-campus Teaching

A key limitation of the current model is that it treats programme structure in a relatively coarse way. In the first-stage scheduling model, overlap penalties are driven by whether two events are associated with the same whole-class cohort, and room feasibility is handled largely through room-type availability. This works reasonably well when students belong to a single department or campus. However, it is less satisfactory for joint degree programmes, where teaching responsibilities, student movement, and timetable priorities are shared across multiple campuses.

A student on a joint pathway may take core events from two subject areas, potentially taught on different campuses. In the current model, these students are only represented indirectly, through the overlap structure induced by the event-programme relationships. As a result, the model can miss some of the real operational difficulty of scheduling joint programmes.

A second limitation is that the current clash treatment is essentially binary: two events either overlap and incur a penalty, or they do not. In practice, conflicts are often matters of degree. An overlap affecting a small optional subgroup should not carry the same weight as one affecting the entire cohort of a joint degree. Similarly, two events on different campuses with a short turnaround are not directly infeasible in the way a same-slot clash is, but they may still be highly undesirable for the students involved. The present model does not capture these distinctions, and a more granular conflict weighting would better reflect the timetabling problem.

12.2 How the Model Could be Adapted in Future Work

A natural next step would be to replace the current coarse event–programme representation with a more explicit *student-sharing* representation. Instead of using only a binary indicator for whether an event belongs to a programme, one could define a parameter such as a_{eh} to denote the number, or proportion, of students in pathway h who attend event e, where h ranges over more refined cohorts such as single-honours pathways, joint degree pathways, or even year-specific route combinations. The overlap weight between two events could then be based on the size of the student population genuinely shared by both events, rather than on a simpler whole-class indicator. For instance, one could define an overlap weight of the form $\omega_{ee'} = \sum_h \min(a_{eh}, a_{e'h})$, or some weighted variant of this, so that the model distinguishes between a clash affecting many joint-degree students and one affecting only a small optional subgroup.

This would improve our model in two ways. First, it would allow the optimization to reflect actual student impact more faithfully. Second, it would reduce the dependence on departmental boundaries,

since the shared-student structure would be determined directly by pathway data rather than inferred from which academic unit "owns" an event. In effect, the model would move from a department-centred view of clashes to a student-centred view.

There is also a data issue. To support joint degree programmes properly, the model would need more detailed information than a standard event list and room inventory. In particular, one would need reliable pathway-level student membership data, identification of which events are compulsory for which subgroups, and reasonable estimates of cross-campus travel times. Given the memory issues we ran into with our current MIP formulation, we caution that this would require significantly more memory than our current model and even for the smaller campus problems may lead to crashes like we saw for the University wide programme we tested.

From an evaluation perspective, this limitation should not be viewed as a failure of the project so much as a boundary on what the current abstraction can represent. The model succeeds in producing feasible and structured schedules under substantial operational constraints, and it provides a defensible basis for large-scale timetable construction. However, the analysis of joint degree programmes shows that there is an important gap between *institutionally feasible* timetables and *student-centred* timetables. Future work should therefore focus on richer cohort modelling, explicit campus-travel considerations, and a tighter link between scheduling and room allocation.

12.3 Whether the Timetabling Team Should Adopt our Methods

We recommend that the timetabling team consider adopting a MIP-based approach for university timetabling, particularly for small to medium-scale instances or decomposed subproblems. The MIP consistently produced high-quality, feasible timetables with strong performance across all evaluation metrics. In contrast, the simulated annealing (SA) approach did not provide comparable solution quality or reliability in our experiments, particularly in maintaining feasibility with respect to key constraints. As such, we do not recommend its use in its current form for operational timetabling.

A key limitation of the MIP approach is the technical expertise required to implement, maintain, and extend the model. It relies on a complex set of constraints and requires familiarity with mathematical optimization, modelling frameworks such as Pyomo, and solvers such as Gurobi. Adapting the model to incorporate new institutional constraints or policy changes would therefore require personnel with sufficient programming and optimization expertise. A further limitation concerns scalability. While the MIP performs well on decomposed or campus-level instances, the full university-wide problem may become computationally intractable depending on available hardware resources.

13 Conclusion: The Success of this Project

At the outset of this project, we set out to address two key questions posed by the timetabling team:

- Can teaching hours be reduced from 9am-6pm to 9am-5pm, Monday-Friday?
- Can teaching on Friday afternoons be eliminated?

Our results show that across the BioQuarter, Easter Bush and King's Buildings campuses, not only is each of these changes individually feasibly, but both can be implemented simultaneously. While we were unable to confirm this at a full University-wide scale, this reflects computational limitations rather than a failure of the approach.

This should not be viewed as a shortcoming of the project. From the outset, we anticipated that the University-wide problem would be extremely difficult to solve directly. By decomposing the problem by campus, we obtained solutions that are both tractable and practically informative. These results demonstrate that substantial timetable compression is achievable in realistic settings, while allowing us to quantify the resulting impact on student experience.

Whether such changes should be implemented remains a broader institutional question. However, from an operational research perspective, our findings provide clear evidence that these proposals are feasible and offer a credible foundation for further consideration by the University.

References

[1] Esraa A. Abdelhalim and Ghada A. El Khayat. A utilization-based genetic algorithm for solving the university timetabling problem (uga). *Alexandria Engineering Journal*, 55(2):1395–1409, 2016. ISSN 1110-0168. doi: https://doi.org/10.1016/j.aej.2016.02.017. URL https://www.sciencedirect.com/science/article/pii/S1110016816000703.

[2] Marco Antonio Aguirre Lam, Fausto Balderas, Nelson Rangel-Valdez, Jose A. Martinez-Flores, Alan G. Aguirre L, and José A. Pérez-Vázquez. University timetabling problem: An analysis of the state of the art. *International Journal of Combinatorial Optimization Problems and Informatics*, 16(3):489–498, Jul. 2025. doi: 10.61467/2007.1558.2025.v16i3.852. URL https://ijcopi.org/ojs/article/view/852.

[3] E. Burke, K. Jackson, J. H. Kingston, and R. Weare. Automated university timetabling: The state of the art. *The Computer Journal*, 40(9):565–571, 1997. doi: 10.1093/comjnl/40.9.565.

[4] V. Cacchiani, A. Caprara, R. Roberti, and P. Toth. A new lower bound for curriculum-based course timetabling. *Computers Operations Research*, 40(10):2466–2477, 2013. ISSN 0305-0548. doi: https://doi.org/10.1016/j.cor.2013.02.010. URL https://www.sciencedirect.com/science/article/pii/S0305054813000506.

[5] Dennis Søren Holm. *Mixed Integer Programming for University Timetabling*. Phd thesis, Technical University of Denmark, Lyngby, Denmark, 2022.

[6] Sadaf Naseem Jat and Shengxiang Yang. A guided search non-dominated sorting genetic algorithm for the multi-objective university course timetabling problem. In Peter Merz and Jin-Kao Hao, editors, *Evolutionary Computation in Combinatorial Optimization*, pages 1–13, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg. ISBN 978-3-642-20364-0.

[7] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983. doi: 10.1126/science.220.4598.671. URL https://www.science.org/doi/abs/10.1126/science.220.4598.671.

[8] J. Kusiak, D. Szeliga, and Ł. Sztangret. 3 - modelling techniques for optimizing metal forming processes. In Jianguo Lin, Daniel Balint, and Maciej Pietrzyk, editors, *Microstructure Evolution in Metal Forming Processes*, Woodhead Publishing Series in Metals and Surface Engineering, pages 35–66. Woodhead Publishing, 2012. ISBN 978-0-85709-074-4. doi: https://doi.org/10.1533/9780857096340.1.35. URL https://www.sciencedirect.com/science/article/pii/B978085709074450003X.

[9] Gerald Lach and Marco Lübbecke. Curriculum based course timetabling: New solutions to udine benchmark instances. *Annals of Operations Research*, 194:255–272, 04 2012. doi: 10.1007/s10479-010-0700-7.

[10] R. Lewis and J. Thompson. Analysing the effects of solution space connectivity with an effective metaheuristic for the course timetabling problem. *European Journal of Operational Research*, 240(3):637–648, 2015. ISSN 0377-2217. doi: https://doi.org/10.1016/j.ejor.2014.07.041. URL https://www.sciencedirect.com/science/article/pii/S0377221714006079.

[11] Mehrnaz Shirani LIRI and Meysam Shahvali KOHSHORI. Multi population hybrid genetic algorithms for university course timetabling. *Analele Universității "Dunărea de Jos" Galați. Fascicula I, Economie și informatica aplicata*, 1(2):5–16, 2012. ISSN 1584-0409.

[12] Barry McCollum, Andrea Schaerf, Ben Paechter, Paul McMullan, Rhyd Lewis, Andrew J. Parkes, Luca Di Gaspero, Rong Qu, and Edmund K. Burke. Setting the research agenda in automated timetabling: The second international timetabling competition. *INFORMS Journal on Computing*, 22(1):120–130, 2010. doi: 10.1287/ijoc.1090.0320. URL https://doi.org/10.1287/ijoc.1090.0320.

[13] Tomáš Müller, Hana Rudová, and Zuzana Müllerová. Real-world university course timetabling at the International Timetabling Competition 2019. *Journal of Scheduling*, 28(2):247–267, 2025. doi: 10.1007/s10951-023-00801-w.

[14] Sadaf Naseem Jat. Genetic algorithms for university course timetabling problems, 2012.

[15] Efstratios Rappos, Eric Thiémard, Stephan Robert, and Jean-François Hêche. A mixed-integer programming approach for solving university course timetabling problems. *Journal of scheduling*, 25(4):391–404, 2022. ISSN 1094-6136.

[16] Kadri Sylejmani, Elton Gashi, and Arbër Ymeri. Simulated annealing with penalization for university course timetabling. *Journal of Scheduling*, 26(5):497–517, 2023. doi: 10.1007/s10951-022-00747-5. URL https://doi.org/10.1007/s10951-022-00747-5.

[17] Jacques Teghem. Metaheuristics. from design to implementation. *European Journal of Operational Research*, 205(2):486–487, 2010. ISSN 0377-2217. doi: https://doi.org/10.1016/j.ejor.2009.12.010. URL https://www.sciencedirect.com/science/article/pii/S0377221709009382.

[18] Tuba Tezer, Ramazan Yaman, and Gülşen Yaman. Evaluation of approaches used for optimization of stand-alone hybrid renewable energy systems. *Renewable and Sustainable Energy Reviews*, 73:840–853, 2017. ISSN 1364-0321. doi: https://doi.org/10.1016/j.rser.2017.01.118. URL https://www.sciencedirect.com/science/article/pii/S1364032117301272.

[19] University of Edinburgh. Shared academic timetabling policy. Technical report, University of Edinburgh, 2018. URL https://edwebcontent.ed.ac.uk/sites/default/files/atoms/files/timetabling_policy_0.pdf. Version 4, approved by CSPC 31/05/2018. Accessed: 2026-04-03.

[20] University of Edinburgh Registry Services. Location modelling, 2024. URL https://registryservices.ed.ac.uk/timetabling-examinations/timetabling/modelling-and-reporting/modelling/location-modelling. Accessed: 2026-04-03.

Appendices

A Mixed Integer Programming Problem Formulation

We consider a two-stage optimization model for generic event scheduling and room assignment over a planning horizon. In the first stage, each event is assigned a common day–start-time pattern across all of its active weeks. In the second stage, each scheduled in-person event occurrence is assigned to a compatible room.

A.1 Stage 1: Event-to-timeslot assignment

We first define the necessary **sets and indices**:

- $\mathcal{E}$: set of events
- $\mathcal{W}$: set of weeks in the planning horizon
- $\mathcal{W}_e \subseteq \mathcal{W}$: set of weeks in which event e occurs
- $\mathcal{D}$: set of days
- $\mathcal{S}_d$: set of timeslots available on day d
- $\mathcal{S}_{ed}^{\text{start}} \subseteq \mathcal{S}_d$: feasible start slots for event e on day d
- $\mathcal{P}$: set of programmes / student cohorts
- $\mathcal{EP} \subseteq \mathcal{E} \times \mathcal{P}$: set of event–programme pairs
- $\mathcal{R}^{\text{type}}$: set of enforced room types
- $\mathcal{K} \subseteq \mathcal{E} \times \mathcal{E}$: set of event pairs with shared whole-class cohorts and common active weeks
- $\Lambda_{ee'}$: set of overlapping day–start combinations for event pair (e, e')
- $\mathcal{C}_{wdtr}$: set of event starts that occupy week w, day d, slot t and require room type r

We now define the **parameters**:

- L_e: duration of event e in timeslots
- ρ_e: room type required by event e
- A_r: number of available rooms of type r
- $g_{ep} = \begin{cases} 1, & \text{if event } e \text{ is whole-class for programme } p \\ 0, & \text{otherwise} \end{cases}$
- $\omega_{ee'}$: overlap penalty weight for event pair (e, e'), given by

$$\omega_{ee'} = \left|\left\{p \in \mathcal{P} : g_{ep} = 1,\ g_{e'p} = 1\right\}\right| \cdot |\mathcal{W}_e \cap \mathcal{W}_{e'}|$$

- $\Lambda_{ee'}$: set of all triples (d, s, s') such that events e and e' would overlap in time if they were scheduled on day d with start slots s and s', that is

$$\Lambda_{ee'} = \left\{(d, s, s') : d \in \mathcal{D},\ s \in \mathcal{S}_{ed}^{\text{start}},\ s' \in \mathcal{S}_{e'd}^{\text{start}},\ [s, s + L_e - 1] \cap [s', s' + L_{e'} - 1] \neq \emptyset\right\}$$

- $\mathcal{C}_{wdtr}$: set of all (e, s) such that $w \in \mathcal{W}_e$, $\rho_e = r$, $s \in \mathcal{S}_{ed}^{\text{start}}$, and event e occupies slot t if it starts at s

The **decision variables** are:

- $x_{ewds} = \begin{cases} 1, & \text{if event } e \text{ starts in week } w, \text{ day } d, \text{ slot } s \\ 0, & \text{otherwise} \end{cases}$
- $y_{eds} = \begin{cases} 1, & \text{if event } e \text{ chooses day } d \text{ and start slot } s \text{ as its common pattern} \\ 0, & \text{otherwise} \end{cases}$

A.1.1 Objective function

The first-stage objective is to minimise weighted overlaps between events that share whole-class cohorts. In the notebook, this is modelled *directly* as a quadratic objective:

$$\min \sum_{(e,e')\in\mathcal{K}} \omega_{ee'} \sum_{(d,s,s')\in\Lambda_{ee'}} y_{eds}\, y_{e'ds'}. \tag{1}$$

This is the exact objective used in the code: each product $y_{eds}y_{e'ds'}$ contributes a penalty whenever two conflicting event patterns are simultaneously selected.

A.1.2 Constraints

Each event must choose exactly one day–start-time pattern:

$$\sum_{d\in\mathcal{D}} \sum_{s\in\mathcal{S}_{ed}^{\text{start}}} y_{eds} = 1, \qquad \forall e \in \mathcal{E}. \tag{2}$$

The chosen pattern must be used in every week in which the event occurs:

$$x_{ewds} = y_{eds}, \qquad \forall e \in \mathcal{E},\ \forall w \in \mathcal{W}_e,\ \forall d \in \mathcal{D},\ \forall s \in \mathcal{S}_{ed}^{\text{start}}. \tag{3}$$

For each week, day, timeslot and room type, the number of simultaneously running events requiring that room type cannot exceed the number of available rooms of that type:

$$\sum_{(e,s)\in\mathcal{C}_{wdtr}} x_{ewds} \le A_r, \qquad \forall w \in \mathcal{W},\ \forall d \in \mathcal{D},\ \forall t \in \mathcal{S}_d,\ \forall r \in \mathcal{R}^{\text{type}}. \tag{4}$$

Finally, the binary restrictions are

$$x_{ewds},\ y_{eds} \in \{0,1\}. \tag{5}$$

Remark. Since the objective in Equation 1 contains bilinear terms in binary variables, the first-stage model is a quadratic binary optimization problem rather than a purely linear mixed-integer program.

A.2 Stage 2: Room assignment

Once the event schedule has been fixed, the second stage assigns each in-person scheduled occurrence to a compatible physical room.

We define the **sets and indices**:

- $\mathcal{O}$: set of scheduled event occurrences, where each occurrence is indexed by $o = (e, w, d, s)$
- $\mathcal{R}$: set of rooms
- $\mathcal{A} \subseteq \mathcal{O} \times \mathcal{R}$: set of feasible occurrence–room assignments
- $\mathcal{H}$: set of occupied time indices (w, d, t)
- $\mathcal{O}_{wdt} \subseteq \mathcal{O}$: set of occurrences occupying week w, day d, slot t

The **parameters** are:

- q_o: size of occurrence o
- ℓ_o: duration of occurrence o in timeslots
- C_r: capacity of room r
- $\kappa_{or} = \begin{cases} 1, & \text{if room } r \text{ is compatible with occurrence } o \\ 0, & \text{otherwise} \end{cases}$

- c_{or}: penalty for assigning occurrence o to room r, defined by

$$c_{or} = \big(\max\{0,\ q_o - C_r\}\big)^2$$

The **decision variable** is:

- $z_{or} = \begin{cases} 1, & \text{if occurrence } o \text{ is assigned to room } r \\ 0, & \text{otherwise} \end{cases}$

A.2.1 Objective function

The room-assignment objective is to minimise the total squared room-capacity shortfall:

$$\min \sum_{(o,r)\in\mathcal{A}} c_{or}\, z_{or}. \tag{6}$$

Although the penalty is squared, this remains a linear objective because c_{or} is precomputed as a parameter for each feasible occurrence–room pair.

A.2.2 Constraints

Each occurrence must be assigned exactly one feasible room:

$$\sum_{r\in\mathcal{R}:(o,r)\in\mathcal{A}} z_{or} = 1, \qquad \forall o \in \mathcal{O}. \tag{7}$$

No room can host more than one occurrence in the same occupied slot:

$$\sum_{o\in\mathcal{O}_{wdt}:(o,r)\in\mathcal{A}} z_{or} \le 1, \qquad \forall (w,d,t) \in \mathcal{H},\ \forall r \in \mathcal{R}. \tag{8}$$

Finally,

$$z_{or} \in \{0,1\}, \qquad \forall (o,r) \in \mathcal{A}. \tag{9}$$

B Simulated Annealing Formulation

The simulated annealing heuristic follows the same two-stage decomposition and uses the same sets, parameters and objective functions as the MIP formulation in Appendix A. We describe below the solution representation, neighbourhood structure, objective evaluation and cooling schedule for each stage. Standard SA concepts (Metropolis criterion, geometric cooling) are assumed known. See [7] for reference.

B.1 Stage 1: Event-to-timeslot assignment

B.1.1 Solution representation

A solution to the time-assignment stage is a mapping

$$\sigma : \mathcal{E} \to \bigcup_{d\in\mathcal{D}} \{d\} \times \mathcal{S}_{ed}^{\text{start}},$$

assigning each event e a day–start-slot pair (d, s). This is equivalent to selecting, for each event, exactly one $y_{eds} = 1$ in the MIP formulation. Weekly assignments are implied: event e is scheduled at (d, s) in every week $w \in \mathcal{W}_e$.

B.1.2 Initial solution

The initial solution $\sigma^{(0)}$ is generated by assigning each event a day–start-slot pair chosen uniformly at random from its feasible set $\{(d, s) : d \in \mathcal{D},\ s \in \mathcal{S}_{ed}^{\text{start}}\}$. Events for which this set is empty under a given scenario (e.g. events whose duration exceeds the available teaching window on every day) are assigned a placeholder slot and flagged as infeasible.

B.1.3 Neighbourhood

The neighbourhood $\mathcal{N}(\sigma)$ consists of all solutions obtainable by changing the assignment of a single event. At each iteration, an event e is selected uniformly at random, and a new pair (d', s') is drawn uniformly from $\{(d, s) : d \in \mathcal{D},\, s \in \mathcal{S}_{ed}^{\text{start}}\}$. The candidate solution σ' agrees with σ on all events except e, for which $\sigma'(e) = (d', s')$.

B.1.4 Objective function

The Stage 1 objective combines the MIP clash penalty (Equation 1) with a weighted room-type violation count:

$$f_1(\sigma) \;=\; \underbrace{\sum_{(e,e')\in\mathcal{K}} \omega_{ee'} \cdot \mathbf{1}\big[\sigma(e) \text{ and } \sigma(e') \text{ overlap}\big]}_{\text{clash penalty (same as MIP)}} \;+\; w_{\text{viol}} \cdot V(\sigma), \tag{10}$$

where the overlap indicator is

$$\mathbf{1}\big[\sigma(e) \text{ and } \sigma(e') \text{ overlap}\big] = \begin{cases} 1, & \text{if } d_e = d_{e'} \text{ and } [\,s_e,\, s_e + L_e - 1\,] \cap [\,s_{e'},\, s_{e'} + L_{e'} - 1\,] \neq \emptyset, \\ 0, & \text{otherwise,} \end{cases}$$

and $V(\sigma)$ counts aggregate room-type capacity violations:

$$V(\sigma) = \sum_{w\in\mathcal{W}} \sum_{d\in\mathcal{D}} \sum_{t\in\mathcal{S}_d} \sum_{r\in\mathcal{R}^{\text{type}}} \max\Big\{0,\; \big|\{e : (e,s) \in \mathcal{C}_{wdtr},\, \sigma(e) = (d,s)\}\big| - A_r\Big\}.$$

The weight w_{viol} was set to 1000 in all experiments. This penalty serves as a surrogate for the hard constraint (4) in the MIP.

B.1.5 Incremental evaluation

Rather than recomputing $f_1(\sigma')$ from scratch, we evaluate the change $\Delta f_1 = f_1(\sigma') - f_1(\sigma)$ by considering only the clash pairs $\mathcal{K}_e = \{(e, e') \in \mathcal{K}\} \cup \{(e', e) \in \mathcal{K}\}$ involving the moved event e. For each pair, we subtract any penalty contributed by the old assignment and add any penalty from the new assignment. This reduces the per-iteration cost from $O(|\mathcal{K}|)$ to $O(|\mathcal{K}_e|)$.

B.1.6 Acceptance criterion and cooling

A candidate move with objective change Δf_1 is accepted with probability

$$P(\text{accept}) = \begin{cases} 1, & \text{if } \Delta f_1 \leq 0, \\ \exp\big(-\Delta f_1 / T_k\big), & \text{otherwise,} \end{cases} \tag{11}$$

where T_k is the temperature at iteration k. The temperature follows a geometric cooling schedule $T_{k+1} = \alpha\, T_k$ with initial temperature T_0 and cooling rate α. To mitigate premature convergence, a reheat mechanism is applied: if the best-known objective does not improve for n_{pat} consecutive iterations, the temperature is increased to $\min(\beta \cdot T_k,\, T_0)$, where β is the reheat factor. The algorithm terminates when $T_k < T_{\min}$, the iteration limit is reached, or the wall-clock time limit expires.

B.1.7 Hyperparameters

The Stage 1 hyperparameters used in the experiments are summarised in Table 6.

Table 6: Stage 1 SA hyperparameters.

Parameter	Symbol	Value
Initial temperature	T_0	100
Minimum temperature	$T_{\min}$	0.01
Cooling rate	α	0.99995
Maximum iterations	$N_{\max}$	500 000 (up to 5 000 000 for large instances)
Reheat patience	n_{pat}	20 000
Reheat factor	β	2.0
Room violation weight	w_{viol}	1 000
Wall-clock time limit		120 s (up to 1 200 s for large instances)

B.2 Stage 2: Room assignment

B.2.1 Solution representation

A solution to the room-assignment stage is a mapping

$$\pi : \mathcal{O} \to \mathcal{R},$$

assigning each scheduled occurrence $o \in \mathcal{O}$ a compatible room r such that $(o, r) \in \mathcal{A}$.

B.2.2 Initial solution (greedy best-fit decreasing)

The initial assignment $\pi^{(0)}$ is constructed by a greedy heuristic. Occurrences are sorted by event size q_o in descending order. Each occurrence is then assigned to the smallest compatible room that does not create a time conflict with any previously assigned occurrence in the same week, day and overlapping slots. If no conflict-free room is available, the occurrence is assigned to the compatible room with the smallest capacity shortfall.

B.2.3 Neighbourhood

At each iteration, an occurrence o is selected uniformly at random and its room is replaced by a different compatible room r' drawn uniformly from $\{r \in \mathcal{R} : (o, r) \in \mathcal{A},\ r \neq \pi(o)\}$.

B.2.4 Objective function

The Stage 2 objective combines the squared capacity shortfall from the MIP (Equation 6) with a double-booking penalty:

$$f_2(\pi) \;=\; \sum_{o \in \mathcal{O}} c_{o,\pi(o)} \;+\; w_{\text{db}} \cdot B(\pi), \tag{12}$$

where $c_{or} = (\max\{0,\, q_o - C_r\})^2$ is the same squared shortfall penalty as in the MIP, and $B(\pi)$ counts the number of double bookings:

$$B(\pi) = \big|\big\{(o, o') : o \neq o',\ \pi(o) = \pi(o'),\ o \text{ and } o' \text{ overlap in time}\big\}\big|.$$

The weight $w_{\text{db}} = 10\,000$ ensures that double bookings are strongly discouraged. In the MIP, double bookings are prevented by the hard constraint (8); here they are penalised because the greedy initialisation and single-swap moves do not guarantee conflict-free solutions at every iteration.

B.2.5 Acceptance criterion and cooling

The Metropolis criterion and geometric cooling follow the same scheme as Stage 1 (Equation 11), with separate hyperparameters given in Table 7.

Table 7: Stage 2 SA hyperparameters.

Parameter	Symbol	Value
Initial temperature	T_0	50
Minimum temperature	$T_{\min}$	0.001
Cooling rate	α	0.99995
Maximum iterations	$N_{\max}$	300 000
Double-booking weight	w_{db}	10 000
Wall-clock time limit		100 s (up to 600 s for large instances)

B.3 Relationship to the MIP formulation

The SA heuristic was designed to be as close to the MIP as possible in order to enable direct comparison of solution quality. The key correspondences and differences are:

1. The Stage 1 clash objective (Equation 10) evaluates the same quantity as the MIP objective (Equation 1), but replaces the sum over all triples in $\Lambda_{ee'}$ with an indicator that checks overlap for the single assigned pair.
2. The MIP enforces room-type capacity (Equation 4) as a hard constraint. The SA penalises violations via the weighted term $w_{\text{viol}} \cdot V(\sigma)$, which cannot guarantee feasibility. This is a known limitation of single-move SA on tightly constrained problems.
3. The Stage 2 capacity objective (Equation 12) uses the same squared shortfall penalty c_{or} as the MIP (Equation 6). The additional double-booking term $w_{\text{db}} \cdot B(\pi)$ replaces the hard no-clash constraint (Equation 8).
4. The MIP explores the feasible region via branch-and-bound with LP relaxations; the SA traverses it through local moves with Metropolis acceptance. The MIP can certify optimality; the SA cannot.

C Teaching Distribution Tables

Table 8 and Table 9 report the percentage of teaching in the 5-6 pm slot and the Friday afternoon teaching load respectively, broken down by campus and scenario. These supplement the summary discussion in Section 7.2.

Table 8: Percentage of teaching scheduled in the 5–6 pm slot by campus and scenario (60-minute slots).

	Easter Bush		BioQuarter		KB	
Scenario	Sem 1	Sem 2	Sem 1	Sem 2	Sem 1	Sem 2
Baseline (9–6, Fri afternoon)	9.79	6.56	1.76	8.72	10.30	10.38
Short Friday (9–6, no Fri afternoon)	5.47	6.17	4.19	0.19	8.74	6.92

Scenarios with a 9 am–5 pm teaching day are omitted since the 5–6 pm slot does not exist under those configurations and the percentage is zero by construction.

Table 9: Friday afternoon teaching load (number of events scheduled after 12 pm on Fridays) by campus and scenario (60-minute slots).

	Easter Bush		BioQuarter		KB	
Scenario	Sem 1	Sem 2	Sem 1	Sem 2	Sem 1	Sem 2
Baseline (9–6, Fri afternoon)	250	189	94	130	1,771	1,973
Reduced core hours (9–5, Fri afternoon)	288	188	73	103	1,881	1,469

Scenarios with no Friday afternoon teaching are omitted since the Friday afternoon load is zero by construction under those configurations.

D Additional Tables

Table 10: MIP results under standard core hours (9 am–6 pm, Friday afternoon teaching permitted), 60-minute slots.

	Programme-level clashes		Capacity deviation		Overcrowded events		Average room utilisation (%)	
Campus	Sem 1	Sem 2	Sem 1	Sem 2	Sem 1	Sem 2	Sem 1	Sem 2
Easter Bush	0	0	1,227	2,231	52	46	69.58	76.76
BioQuarter	36	0	800	654	51	29	64.78	60.14
KB – chem/phys	111	585	2,057	2,388	39	39	61.60	62.04
KB – eng/geo/bio	66	106	5,324	11,806	273	325	64.51	63.74
KB – maths	178	776	32,829	22,889	389	279	53.78	54.86
KB – Total	355	1,467	40,210	37,083	701	643	60.37	60.44
KB (Alt. Room Assignment)	–	–	4,476	5,864	124	136	57.50	57.32

Table 11: MIP results under reduced core hours (9 am–5 pm, Friday afternoon teaching permitted), 60-minute slots.

	Programme-level clashes		Capacity deviation		Overcrowded events		Average room utilisation (%)	
Campus	Sem 1	Sem 2	Sem 1	Sem 2	Sem 1	Sem 2	Sem 1	Sem 2
Easter Bush	0	0	1,244	2,312	48	47	69.09	77.40
BioQuarter	55	0	805	788	52	32	65.73	59.01
KB – chem/phys	224	836	2,137	2,721	38	62	61.27	62.57
KB – eng/geo/bio	203	208	6,349	6,151	297	293	65.64	63.87
KB – maths	383	1,026	34,638	26,418	404	319	54.80	55.69
KB – Total	810	2,070	43,124	35,290	739	674	61.12	60.89
KB (Alt. Room Assignment)	–	–	4,494	3,861	136	119	58.26	57.61

Table 12: MIP results under reduced Friday afternoon (9 am–6 pm, no teaching after 12 pm on Fridays), 60-minute slots.

	Programme-level clashes		Capacity deviation		Overcrowded events		Average room utilisation (%)	
Campus	Sem 1	Sem 2	Sem 1	Sem 2	Sem 1	Sem 2	Sem 1	Sem 2
Easter Bush	3	0	1,469	2,243	64	48	70.20	78.06
BioQuarter	59	2	873	737	61	26	66.91	57.72
KB – chem/phys	269	919	2,039	2,530	30	74	61.58	62.41
KB – eng/geo/bio	194	325	10,349	7,143	357	290	65.93	63.55
KB – maths	438	1,076	35,097	24,775	421	282	55.61	55.24
KB – Total	901	3,848	47,485	34,448	808	646	61.59	60.56
KB (Alt. Room Assignment)	–	–	5,030	4,497	161	133	58.05	57.79

Table 13: MIP results under both reduced core hours and reduced Friday afternoon (9 am–5 pm, no teaching after 12 pm on Fridays), 60-minute slots.

	Programme-level clashes		Capacity deviation		Overcrowded events		Average room utilisation (%)	
Campus	Sem 1	Sem 2	Sem 1	Sem 2	Sem 1	Sem 2	Sem 1	Sem 2
Easter Bush	3	0	1,319	2,332	55	48	70.44	76.94
BioQuarter	84	2	914	1,278	68	78	65.51	71.64
KB – chem/phys	497	1,235	2,062	2,521	37	66	61.16	61.85
KB – eng/geo/bio	390	629	8,853	13,691	468	358	68.09	66.30
KB – maths	640	1,350	36,495	26,522	420	292	55.68	55.64
KB – Total	1,527	3,214	47,410	42,734	925	716	62.47	61.80
KB (Alt. Room Assignment)	–	–	5,167	5,706	176	182	59.08	58.71

E AI Usage

In preparing this report, we used Claude Code to assist with generating the plots included in the report. In particular, we provided the following prompt:

```
Task: Create 4 publication-quality figures for a university OR report, each containing two
↪  subplots (Semester 1 and Semester 2). The figures compare MIP timetabling results across
↪  four scheduling scenarios for three campus groups: Easter Bush (EB), BioQuarter (BQ),
↪  and Kings Buildings Alternative Room Assignment (KB Alt).

Data source: Use Results.xlsx, Sheet1. The data starts at row 4 (0-indexed row 3 after
↪  skipping 3 header rows). The columns are:
idx, Day Length, Friday Afternoon?, Campus, Slot Duration, Semester, Feasible?,
Overlap Penalty, Overlap Optimal?, Capacity Penalty (Linear), Capacity Penalty (Quadratic),
Capacity Optimal?, Average Room Utilisation, Number of capacity violations, Memory Run Out?,
Weighted Room Utilisation, Capacity Violation Seats, Assigned Events, Unknown Capacity
↪  Events,
Average Spare Capacity, Maximum Capacity Violation, Unassigned Events

Drop the first unnamed index column. Use only 60-minute slot rows (Slot Duration == 60).

Four scenarios (grouped bars within each campus):
1. Baseline: Day Length == '9-6', Friday Afternoon? == 'Y'
2. Reduced Core Hours: Day Length == '9-5', Friday Afternoon? == 'Y'
3. Short Friday: Day Length == '9-6', Friday Afternoon? == 'N'
4. Both: Day Length == '9-5', Friday Afternoon? == 'N'


Three campus groups on the x-axis of each subplot:
• Easter Bush: Campus == 'Easter Bush'
• BioQuarter: Campus == 'BioQuarter'
 KB Alt: Campus == 'KB1 (Alternative Room Assignment)' for Sem 1, Campus == 'KB2
↪  (Alternative Room Assignment)' for Sem 2

Special case for KB Alt programme-level clashes: KB Alt does not have its own clash values
↪  (it reuses the Stage 1 time assignment from KB-Total). For the programme-level clashes
↪  plot only, use the Overlap Penalty value from Campus == 'KB1 - Total' (Sem 1) and Campus
↪  == 'KB2 - Total' (Sem 2) for the KB Alt bars. For all other metrics, use the KB Alt
↪  row's own values.

Four figures, one per metric:
1. Programme-level clashes - column: Overlap Penalty
2. Capacity deviation - column: Capacity Penalty (Linear)
3. Overcrowded events - column: Number of capacity violations
4. Average room utilisation (%) - column: Average Room Utilisation

Layout for each figure:
• Two subplots side by side: left = Semester 1, right = Semester 2
• Three groups on the x-axis: Easter Bush, BioQuarter, KB Alt
```

```
• Four bars per group, one per scenario
• Shared y-axis label, individual subplot titles ("Semester 1", "Semester 2")
• Single shared legend at the bottom or top of the figure

Style requirements (academic/publication standard):
• Use matplotlib with a clean style (seaborn-v0_8-whitegrid or similar)
 Muted, colourblind-friendly colour palette (e.g. from seaborn's colorblind palette or
↪  manually chosen)
• No background gridlines on x-axis, light horizontal gridlines only
• Axis labels in regular weight, title in bold
• Bar edge colour: thin dark grey outlines
• Font: sans-serif (e.g. DejaVu Sans or Helvetica), size 10-11 for labels, 9 for tick labels
• Figure size: approximately 10 × 4 inches per figure
 Add value labels on top of each bar (formatted as integers for counts, one decimal for
↪  percentages)
 Scenario labels in legend: "96 Fri Afternoon", "95 Fri Afternoon", "96 Short Friday", "95
↪  Short Friday"
• Save each figure as both PDF and PNG (300 dpi) with tight bounding box

Output filenames:
• plot_clashes.pdf / plot_clashes.png
• plot_capacity_deviation.pdf / plot_capacity_deviation.png
• plot_overcrowded.pdf / plot_overcrowded.png
• plot_utilisation.pdf / plot_utilisation.png
\end{verbatim}
```